Day 5, SPb SU LOUD Enough Contest 2

February 4, 2023

Greedy Bipartite Matching

- We want to find a greedy matching in a bipartite graph.
- Greedy matching is a matching that maximizes number of edges of weight 1, then number edges of weight 2, etc.
- We get edges in order of increasing of their weights.
- Now, how to solve the problem? Our plan is to solve our problem for edges of weight 1 first and then modify the graph in a way such that solving it for heavier edges won't break anything.

Greedy Bipartite Matching

- Consider a vertex cover on our light edges (initially, edges of weight 1).
- Let L_1, R_1 be the vertices from VC in left and right part correspondingly, and L_2, R_2 be other vertices.
- It is easy to see that light edges of the type L_1–R_1 never belong to the answer. So, we remove them.
- Similarly, heavy edges can only be of the type L_2–R_2. So, we remove others.
- Now, we can also see that any maximal matching in the resulting graph could be transformed into a matching having maximal number of light edges.
- Repeat this process for all q weights.

Greedy Bipartite Matching

- For each weight we need to find a maximal matching and then to remove some edges which can be done in $O(mq)$ total time.
- The naive upper bound is $O(q \cdot \text{Matching}(n, m) + nq)$, which seem to be too much, however, running the optimized version of Kuhn's algorithm was enough to get Accepted.
- Running optimized version of Kuhn at this point of time is a kind of magic that throws against wisdom rather than intelligence, but I'll try to convince you that it clearly fits in TL.

Greedy Bipartite Matching

- We can use online version of slow Kuhn instead. It will work in time $O(nm + nq)$. Note that there is no q in the first term anymore.
- It still a bit too much, but if you look closely, fast Kuhn could be considered as an optimized version of online Kuhn (up to some additional $O(nq)$ overhead, which is small enough), thus, it should be fast enough already.
- Intuitively, the running time of the described algorithm should be $O(\text{Matching}(n, m) + nq)$, but it is hard to prove, as well as it is hard to prove that Kuhn is indeed fast enough for $m = 2 \cdot 10^5$.
- If you are still in doubt, you can implement Dinic' (or Hopcroft–Karp) algorithm with an easy upper bound of $O\left(mq\sqrt{n}\right)$ and a bound of $O\left(m\sqrt{qn}\right)$ that is harder to prove.

Banshee

- We have an army of StarCraft banshees, we want to eliminate all opponent's buildings as quickly as possible.
- All our units are at the origin, all buildings have non-negative coordinates.
- Let's forget about shields for a second. How to solve the problem then?
- First of all, we can see that we can limit our banshees to choose their target in the order of increasing coordinate.
- Now, we can do a binary search on the answer and solve the problem of minimizing the amount of banshees required to destroy all the buildings.
- How do we do that?

Banshee

- Greedy!
- Given the time bound we can see units as buying some number of shots we can distribute on a prefix of buildings.
- Thus, we can consider buildings from right to left and each time choose the smallest number of units we should add in order to have enough unassigned shots to destroy the current building.
- So, we solved a problem without shields in time $O\left(n \log \varepsilon^{-1}\right)$. In this problem $\varepsilon \approx 10^{-15}$.

Banshee

- But what to do with shields?
- Nothing! We can prove that we can construct an optimal (in the sense of the previous slide) solution that does not allow recharging.
- Consider optimal solutions that minimize the number of the furthest building that recharges its shields. Among those, consider a solution that minimizes the number of different units that are hitting it, but then flies further.
- Consider the banshee that kills the building. If the building is its first target or was targeted only by it, then the shields never recharge.
- Otherwise, we can exchange its shots for the previous builds with the shots of another banshee that flies further. It contradicts minimality.

Elimination Race

- We have a results of races. We can reorder them, the last participant eliminated.
- For each participant find whether he can win and how (if he can).
- Let's solve a problem for the first participant. Consider a bipartite graph between races and players other than first where the edge is present iff the first participant beats the player in the race.
- The answer exists iff such graph contains a perfect matching.
- Proof in one direction is obvious: we can match races to the eliminated players.

Elimination Race

- In order to prove into another let's consider a smallest counterexample.
- If there is a race matched with the last player, the counterexample is not smallest.
- Otherwise draw a directed edge from the matched player to the last player in each race.
- We obtained a graph on players with out-degree equal to 1 for each vertex, thus, it contains a cycle. Therefore, we can replace matches according to this cycle and obtain a contradiction.
- Note that this proof also provides us an algorithm to construct a permutation from the matching. It is doable even in linear time, if implemented carefully enough.

Elimination Race

- The overall complexity is $O(n \cdot \text{Matching}(n, n^2))$.
- Again, known upper bounds for Kuhn are not tight and the best known lower bound for $|E| \approx |V|^2$ is $\Omega\left(\frac{|V|^3}{\log|V|}\right)$, so here we appeal to the knowledge how fast it practically works.
- However, there are two key optimizations:
  1. Using Kuhn's algorithm with bitsets. It leads to the complexity of $O\left(\frac{n^4}{w}\right)$.
  2. Reusing a partial matching from one participant to compute answers for another.

  Using any of them could be sufficient to get Accepted, using both of them makes your code fit into a third of TL. Again, you can also use Dinic for better theoretical bounds.

Shared Memory Switch

- We need to construct an optimal algorithm for the shared memory switch.
- Firstly, run the algorithm with infinite B, compute the time each packet was sent.
- Now, we run second time and each time we have to push out a packet, we choose the packet with the largest time of sending. It is always the last element in the queue, so we can store them in a heap. The complexity is $O(n \log n)$.
- Such algorithm is called LateQD, the proof of its optimality is relatively long (but not necessary too hard), it is written in detail in the paper New Competitiveness Bounds for the Shared Memory Switch.
- This algorithm allowed to obtain new lower bounds for competitiveness for the online policy LQD (Longest Queue Drop) which chooses a longest queue whenever is has to push something out.

Text Editor

- The task is to make a simple text editor with operations load, write, print, copy, cut, paste, insert, undo and redo.
- A naive solution is to store strings in a persistent treap and answer each query in logarithmic time.
- It would definitely work, but it is not expected to fit into ML and the bounds for serialization.
- The next step is to make each node "wide" and store a small string there, similarly to structures like B-trees. It may get Accepted, however, there is a bit more elegant solution.
- We can store a concatenation of all input queries in a string separately and make nodes of our treap hold views into this string. Now, we don't have to worry about small nodes eating all our memory.
- This is the authors' solution!

LCSLCSLCS

- The task is to compute LCS for two long strings with short periods.
- A little spoiler: this problem is solvable in time $O(n \log n \log m + n^2)$ where n is the length of our strings and m is the number of repetitions.
- In order to solve this problem, we need a notion of sticky braids that also called seaweed in two (part one, part two) recently published articles on this topic by ko_osaga.
- There is a dynamic programming solution without it, it works in time $O(n^3 \log m)$.
- Now, we will briefly describe some key moments of the solution with sticky braids.

LCSLCSLCS

- First of all, we need to solve a simpler problem of computing LCS for the strings A and B^∞ using sticky braids.
- The result would be a periodic sticky braid (with period B) that we can later use to compute the answer for A and B^k for any k in linear (in $|B|$) time.
- In order to do that, we need to remove all unique letters from both strings and then start computing each new row from the cell with a wall. The complexity of this part is $O(|A||B|)$.
- Now, we only need to learn how to concatenate such periodic braids. If we can do that quickly enough, then the problem could be solved using ordinary binary exponentiation.

LCSLCSLCS

- Our main obstacle is the fact that the braids are infinite. However, if we take three continuous periods and concatenate them like regular braids, the middle period would be valid as a period for the result.
- The main idea of the proof why is to note that if our resulting braid is invalid (thus, some two lines intersect twice), then it should happen for lines from two adjacent periods.
- So, we reduced our problem to the known problem of multiplication of sticky braids (sticky multiplication).

LCSLCSLCS

- Sticky multiplication can be done via tropical multiplication of two unit-Monge matrices in cubic time naively or in quadratic time by using Knuth's optimization.
- The total complexities in those cases are $O(n^3 \log m)$ and $O(n^2 \log m)$, respectively. Practically, the first version is about five times faster than the dp solution and the second solution was used to choose the TL for this problem.
- However, there is an algorithm Steady Ant that does sticky multiplication in time $O(n \log n)$ giving the total complexity for this problem of $O(n \log n \log m + n^2)$.
- The authors are very grateful to Alexander Tiskin for his tremendous help in theoretical side of this problem and also for his great course on string algorithms that was held in SPb SU.

Day 5, SPb SU LOUD Enough Contest 2

February 4, 2023

Courses

- We may write natural dynamic programming $dp[i][j]$ with states - money spend and IQ.

Courses

- We may write natural dynamic programming $dp[i][j]$ with states - money spend and IQ.
- Tranisitions - $dp[i][j] = \sum n[i][j] * dp[i-c][j-d]$. Time is $n^2(\max c)^2$.

Courses

- Let's look to the generating function $F_i(x) = \sum dp[i][j]x^j$

Courses

- Let's look to the generating function $F_i(x) = \sum dp[i][j]x^j$
- We need to calculate sum of some coefficients of F_i for every $i \leq n$.

Courses

- Let's look to the generating function $F_i(x) = \sum dp[i][j]x^j$
- We need to calculate sum of some coefficients of F_i for every $i \leq n$.
- Let's remember the Fourier transform. This sum of coefficients may be rewritten as $\sum_t c_t F_i(\epsilon_t)$, where ϵ_t - roots of unity.

Courses

- Let's look to the generating function $F_i(x) = \sum dp[i][j]x^j$
- We need to calculate sum of some coefficients of F_i for every $i \leq n$.
- Let's remember the Fourier transform. This sum of coefficients may be rewritten as $\sum_t c_t F_i(\epsilon_t)$, where ϵ_t - roots of unity.
- But how the $F_i(\epsilon_t)$ looks like?

Courses

- Consider the generating function $F_i(\epsilon_t)y^i$.

Courses

- Consider the generating function $F_i(\epsilon_t)y^i$.
- It is some rational function with degree of denominator and numerator no more then $\max c$.

Courses

- Consider the generating function $F_i(\epsilon_t)y^i$.
- It is some rational function with degree of denominator and numerator no more then $\max c$.
- We just need to calculate linear combination of such rational functions, it can be done by divide-and-conquer. It is $O(n(\max c)\log^2 n)$.

Young

- We have graph, vertices are Young tableuxes, edge between them - if they differ in one cell. The task is to calculate number of ways to reach one vertex from another in the given number of steps.

Young

- We have graph, vertices are Young tableuxes, edge between them - if they differ in one cell. The task is to calculate number of ways to reach one vertex from another in the given number of steps.
- Lets say that the edge goes up, if it decreases number of cells, else down.

Young

- We have graph, vertices are Young tableuxes, edge between them - if they differ in one cell. The task is to calculate number of ways to reach one vertex from another in the given number of steps.
- Lets say that the edge goes up, if it decreases number of cells, else down.
- Main observation - for every vertex number of edges going up is less by 1 then number of edges going down.

Young

- We have graph, vertices are Young tableuxes, edge between them - if they differ in one cell. The task is to calculate number of ways to reach one vertex from another in the given number of steps.
- Lets say that the edge goes up, if it decreases number of cells, else down.
- Main observation - for every vertex number of edges going up is less by 1 then number of edges going down.
- Moreover, if 2 vertices have common neighbor, they have both common up neighbor and down neighbor.

Young

- Let's call a signature sequence of letters U and D. Path satisfies it, if in letter U it goes up, and in letter D it goes down.

Young

- Let's call a signature sequence of letters U and D. Path satisfies it, if in letter U it goes up, and in letter D it goes down.
- Let's calcualte number of paths of some signature ($F(S)$). Suppose it has two adjacent letters DU in this order (sequence is S_1DUS_2). Then we can see that $F(S) = F(S_1UDS_2) + F(S_1S_2)$.

Young

- Let's call a signature sequence of letters U and D. Path satisfies it, if in letter U it goes up, and in letter D it goes down.
- Let's calcualte number of paths of some signature ($F(S)$). Suppose it has two adjacent letters DU in this order (sequence is S_1DUS_2). Then we can see that $F(S) = F(S_1UDS_2) + F(S_1S_2)$.
- In this manner we can express all $F(S)$ by linear combination of $F(UU\ldots UDD\ldots D)$. Coefficients in expression have combinatorial sence.

Young

- Let's call a signature sequence of letters U and D. Path satisfies it, if in letter U it goes up, and in letter D it goes down.
- Let's calcualte number of paths of some signature ($F(S)$). Suppose it has two adjacent letters DU in this order (sequence is S_1DUS_2). Then we can see that $F(S) = F(S_1UDS_2) + F(S_1S_2)$.
- In this manner we can express all $F(S)$ by linear combination of $F(UU\dots UDD\dots D)$. Coefficients in expression have combinatorial sence.
- Then we can represent sum of all $F(S)$ by linear combination of $F(UU\dots UDD\dots D)$. Coefficients may be found explicitly.

Young

- Values of all $F(UU\ldots UDD\ldots D)$ can be found in dynamic programming, where states are Young subdiagrams of one of the two initial diagrams.

Young

- Values of all $F(UU\ldots UDD\ldots D)$ can be found in dynamic programming, where states are Young subdiagrams of one of the two initial diagrams.
- Asymptotics is $O(k+$"Number of Young subdiagrams of the given one"$)$, which is small enough.

Endless road

- We have 3 guys, every second random one moves 1 step in the positive direction. Task is to calculate expectation of maximal coordinate of them.

Endless road

- We have 3 guys, every second random one moves 1 step in the positive direction. Task is to calculate expectation of maximal coordinate of them.
- This number is indeed $F(n) = \sum_{x+y+z=n} max(x+a, y+b, z+c) \frac{n!}{3^n x! y! z!}$.

Endless road

- We have 3 guys, every second random one moves 1 step in the positive direction. Task is to calculate expectation of maximal coordinate of them.
- This number is indeed $F(n) = \sum_{x+y+z=n} max(x+a, y+b, z+c)\frac{n!}{3^n x!y!z!}$.
- Consider simpler sum: $S(n) = \sum_{x+y+z=n, x+a \geq y+b, x+a \geq z+c} \frac{1}{x!y!z!}$. Logic wih initial one is the same.

Endless road

- We have 3 guys, every second random one moves 1 step in the positive direction. Task is to calculate expectation of maximal coordinate of them.
- This number is indeed $F(n) = \sum_{x+y+z=n} max(x+a, y+b, z+c)\frac{n!}{3^n x!y!z!}$.
- Consider simpler sum: $S(n) = \sum_{x+y+z=n, x+a \geq y+b, x+a \geq z+c} \frac{1}{x!y!z!}$. Logic wih initial one is the same.
- Let's try to calculate $S(n)$ from $S(n-1)$.

Endless road

- Denote $A(x, y, z) = \frac{1}{x!y!z!}$. Then we can see that $nA(x, y, z) = A(x, y, z-1) + A(x, y-1, z) + A(x, y, z-1)$. That is, we can see that if we sum up this equality for whole region, in the right side will be almost $3S(n-1)!$

Endless road

- Denote $A(x, y, z) = \frac{1}{x!y!z!}$. Then we can see that $nA(x, y, z) = A(x, y, z-1) + A(x, y-1, z) + A(x, y, z-1)$. That is, we can see that if we sum up this equality for whole region, in the right side will be almost $3S(n-1)!$
- Problems will be only on borders. So we need to decrease or increase $S(n)$ by some sums where $x + a$ is almost equal $y + b$ or $z + c$.

Endless road

- Denote $A(x, y, z) = \frac{1}{x!y!z!}$. Then we can see that $nA(x, y, z) = A(x, y, z-1) + A(x, y-1, z) + A(x, y, z-1)$. That is, we can see that if we sum up this equality for whole region, in the right side will be almost $3S(n-1)!$
- Problems will be only on borders. So we need to decrease or increase $S(n)$ by some sums where $x + a$ is almost equal $y + b$ or $z + c$.
- It is the sums like $\sum_{x+a \geq y+b} B(x, y)$, where $B(x, y) = \frac{1}{x!(x+t)!y!}$

Endless road

- Denote $A(x, y, z) = \frac{1}{x!y!z!}$. Then we can see that $nA(x, y, z) = A(x, y, z-1) + A(x, y-1, z) + A(x, y, z-1)$. That is, we can see that if we sum up this equality for whole region, in the right side will be almost $3S(n-1)!$
- Problems will be only on borders. So we need to decrease or increase $S(n)$ by some sums where $x + a$ is almost equal $y + b$ or $z + c$.
- It is the sums like $\sum_{x+a \geq y+b} B(x, y)$, where $B(x, y) = \frac{1}{x!(x+t)!y!}$
- How to calculate this? Almost the same! We can express $B(x, y)$ as a linear combination of $B(x-1, y), B(x, y-1), B(x, y-2)$ and pass to the borders, which are where all $x + a, y + b, z + c$ are almost equal, and these are just binomial coefficients.

Endless road

- Denote $A(x, y, z) = \frac{1}{x!y!z!}$. Then we can see that $nA(x, y, z) = A(x, y, z-1) + A(x, y-1, z) + A(x, y, z-1)$. That is, we can see that if we sum up this equality for whole region, in the right side will be almost $3S(n-1)!$
- Problems will be only on borders. So we need to decrease or increase $S(n)$ by some sums where $x + a$ is almost equal $y + b$ or $z + c$.
- It is the sums like $\sum_{x+a \geq y+b} B(x, y)$, where $B(x, y) = \frac{1}{x!(x+t)!y!}$
- How to calculate this? Almost the same! We can express $B(x, y)$ as a linear combination of $B(x-1, y), B(x, y-1), B(x, y-2)$ and pass to the borders, which are where all $x + a, y + b, z + c$ are almost equal, and these are just binomial coefficients.
- Time is $O(n)$ with big constant. This idea works for any number of points.

Sets may be good

- Given a graph, we need to calculate number of sets of vertices with even number of edges between them.

Sets may be good

- Given a graph, we need to calculate number of sets of vertices with even number of edges between them.
- We can look at it as follows: we have some quadratic polynomial $P(x_1, \dots x_n)$ over $\mathbb{F}_2$, we need to calculate number of solutions of equation $F(x_1, \dots, x_n) = 0$.

Sets may be good

- Given a graph, we need to calculate number of sets of vertices with even number of edges between them.
- We can look at it as follows: we have some quadratic polynomial $P(x_1, \dots x_n)$ over $\mathbb{F}_2$, we need to calculate number of solutions of equation $F(x_1, \dots, x_n) = 0$.
- We can suppose F does not contain term x_i^2, cause $x_i^2 = x_i$

Sets may be good

- Given a graph, we need to calculate number of sets of vertices with even number of edges between them.
- We can look at it as follows: we have some quadratic polynomial $P(x_1, \dots x_n)$ over $\mathbb{F}_2$, we need to calculate number of solutions of equation $F(x_1, \dots, x_n) = 0$.
- We can suppose F does not contain term x_i^2, cause $x_i^2 = x_i$
- Let's take one variable x_1. If there is no monomials with it, then in doesn't have any influence to polynomial.
- In the other case, we can say that $P = x_1 L(x_2, \dots x_n) + Q(x_1, \dots, x_n)$, where L is linear.

Sets may be good

- Lets divide all variable substitutions on 2 cases - $L = 0$ and $L = 1$.

Sets may be good

- Lets divide all variable substitutions on 2 cases - $L = 0$ and $L = 1$.
- In the second variant, there is equal number of solutions with $P = 0$ and $P = 1$, cause when we change x_1, P is also changing.

Sets may be good

- Lets divide all variable substitutions on 2 cases - $L = 0$ and $L = 1$.
- In the second variant, there is equal number of solutions with $P = 0$ and $P = 1$, cause when we change x_1, P is also changing.
- So we reduce the problem to the following one: how many solutions of system $P = 0, L = 0$ are.

Sets may be good

- Lets divide all variable substitutions on 2 cases - $L = 0$ and $L = 1$.
- In the second variant, there is equal number of solutions with $P = 0$ and $P = 1$, cause when we change x_1, P is also changing.
- So we reduce the problem to the following one: how many solutions of system $P = 0, L = 0$ are.
- But we can make a variables changing to make L new variable. So we reduced the problem to the same problem with less number of variables. We can continue this process further.

Sets may be good

- Lets divide all variable substitutions on 2 cases - $L = 0$ and $L = 1$.
- In the second variant, there is equal number of solutions with $P = 0$ and $P = 1$, cause when we change x_1, P is also changing.
- So we reduce the problem to the following one: how many solutions of system $P = 0, L = 0$ are.
- But we can make a variables changing to make L new variable. So we reduced the problem to the same problem with less number of variables. We can continue this process further.
- Time is $O(n^3)$.

Day 5, SPbSU LOUD Enough Contest 2

February 4, 2023

Good and Lucky Matrices

- There is always the same number of good and lucky $n \times n$ matrices.
- Construct a good matrix row-by-row.
- Each new row should *not* be in some linear space.
- The total number of good matrices is $(2^n - 2^0) \cdot (2^n - 2^1) \cdot \ldots \cdot (2^n - 2^{n-1})$.
- A lucky matrix: at least one 1 in the first row, the corresponding column can be arbitrary. Repeat on the $(n-1) \times (n-1)$ matrix.
- The total number of lucky matrices is $(2^n - 1) \cdot 2^{n-1} \cdot (2^{n-1} - 1) \cdot 2^{n-2} \cdot \ldots \cdot (2 - 1) \cdot 2^0$.
- These numbers are the same.

Good and Lucky Matrices

- OK, but how to solve the problem?
- An "uglier" way : good matrices $\leftrightarrow$ sequences of binary blocks $\leftrightarrow$ lucky matrices.
- Requires implementing four conversions, but works.
- The intended solution converts good matrices to lucky matrices (and vice versa) directly, so you only need to implement two conversion procedures.

Good and Lucky Matrices

- Good $\to$ lucky: run Gauss, but don't swap the rows.
- Instead, for each row i, find the first yet-unused column j with $A_{i,j} = 1$.
- Now, "freeze" the following items into the answer: $A_{i,k}$ for each unused column k and $A_{r,j}$ for each $r \geqslant i$.
- Now, proceed with the usual step of the Gauss algorithm: ensure that the new values of $A_{r,j}$ are all zero when $r > i$. Notice that we already "froze" these matrix entries into the answer.
- The result is a lucky matrix, with $i \to j$ being exactly the greedy matching.

Good and Lucky Matrices

- Good $\to$ *lucky*: run Gauss, but don't swap the rows.
- Instead, for each row i, find the first yet-unused column j with $A_{i,j} = 1$.
- Now, "freeze" the following items into the answer: $A_{i,k}$ for each unused column k and $A_{r,j}$ for each $r \geqslant i$.
- Now, proceed with the usual step of the Gauss algorithm: ensure that the new values of $A_{r,j}$ are all zero when $r > i$. Notice that we already "froze" these matrix entries into the answer.
- The result is a good matrix, with $i \to j$ being exactly the greedy matching.

Good and Lucky Matrices

- Lucky $\rightarrow$ good: recover the greedy matching and then reverse the above process, starting from the lower rows and then going to the upper ones.
- The process was specifically made to be reversible: we remember $A_{r,j}$ before the row-xor operations, therefore we know whether or not we have actually done them during the Gauss algorithm.

Two Missing Numbers/Nimbers

- There was an unintentional hint: the problem was named "Two Missing Nimbers" in the testing system and in the contest standings.
- Interpret the elements of the input as elements of the field of size 2^{64}.
- This field has several interpretations, they are all isomorphic and have characteristic 2 (meaning that $x + x = 0$ for any x).
- Suppose that the target numbers are x and y.
- The sum of all input numbers is $x + y$ (all other numbers cancel out).
- We can also compute the sum of squares, but it is not useful: $(x + y)^2 = x^2 + 2xy + y^2 = x^2 + y^2$, because $2 = 0$.
- The sum of cubes is better: $(x + y)^3 - (x^3 + y^3) = 3(x^2y + xy^2) = xy(x + y)$.
- Recover xy through division.

Two Missing Numbers/Nimbers

- Alternatively: compute $1/x + 1/y = (x+y)/(xy)$ (zeroes are improbable).
- Or compute $x^2y + xy^2$ directly and not through the sum of cubes: when we add z to the list, this value increases by $z^2 \cdot s + z \cdot s^2$ (again, here we use that $(x_1 + \ldots + x_k)^2 = x_1^2 + \ldots + x_k^2$).
- In the end, we know $x + y$ and xy.
- Need to solve a quadratic equation. There are multiple methods.
- The following should work: to solve $p(x) = 0$, where $p(x) = x^2 + ax + b$, compute the GCD of $(x + r)^{(2^{64}-1)/3} - 1$ and $p(x)$ (r is random here).
- This way, we filter out exactly one third of all non-zero elements of the field.
- A common heuristic suggests that we will recover all roots in $O(1)$ iterations.